

Proyecto: Food Store

Sistema de Gestión de Pedidos

Facultad Regional Mendoza

Tecnicatura Universitaria en Programación

Base de Datos II

Grupo de Trabajo: Franco Sampieri, Mauro Di Pietro, Renzo Victoria, Thomas Martinez, Alam Gonzales

Fecha: 20 de agosto de 2026

Índice

Sistema de Gestión de Pedidos.....	1
Facultad Regional Mendoza.....	1
Tecnatura Universitaria en Programación.....	1
Base de Datos II.....	1
Índice.....	2
Marco Teórico: Base de Datos Relacional.....	3
1. Modelo Relacional.....	3
2. Normalización.....	3
3. Restricciones e Integridad de Datos.....	4
4. Transacciones y Propiedades ACID.....	4
Diagrama.....	5
Preguntas guía.....	5
Decisiones de Diseño.....	6
1. Normalización y Descomposición Relacional.....	6
2. Manejo del Total y Cálculo Automático.....	6
3. Borrado Lógico (Soft Delete).....	7
4. Estrategia de Indexación.....	7
Dificultades, Soluciones y Referencias.....	7
1. Dificultades Encontradas y Soluciones Implementadas.....	7
2. Bibliografía y Webgrafía.....	8
3. Enlaces al Repositorio y Video Demostrativo.....	8

Marco Teórico: Base de Datos Relacional

Fundamentos teóricos y su aplicación en la base de datos FoodStore

1. Modelo Relacional

Propuesto formalmente por Edgar F. Codd en 1970, el **Modelo Relacional** estructura y organiza la información a través de relaciones matemáticas representadas visualmente como *tablas* compuestas por *tuplas* (filas o registros) y *atributos* (columnas o campos). Cada relación define un conjunto ordenado de datos con tipos estrictos, asegurando la separación clara entre la representación lógica y la implementación física del motor de almacenamiento.

Aplicación en FoodStore: El sistema modela el dominio del negocio descomponiéndolo en entidades normalizadas: usuario, categoria, producto, pedido y detalle_pedido. Las dependencias e interacciones entre estas entidades se estructuran mediante asociaciones explícitas fundamentadas en claves primarias y foráneas, garantizando la consistencia y escalabilidad de los datos.

2. Normalización

La normalización es la técnica formal empleada en el diseño de bases de datos relacionales cuyo objetivo principal es **minimizar la redundancia de datos** y erradicar las posibles **anomalías de inserción, actualización y borrado**. El esquema de *FoodStore* cumple con los estándares de la **Tercera Forma Normal (3FN)**:

Forma Normal	Criterio Teórico	Implementación en FoodStore
1FN (Primera FN)	Todos los atributos contienen valores atómicos indivisibles; no existen grupos repetitivos ni atributos multivaluados en las tablas base.	Atributos atómicos directos como precio, stock, mail o nombre_usuario.
2FN (Segunda FN)	Cumple con la 1FN y cada atributo no clave posee dependencia funcional completa de la clave primaria.	En detalle_pedido, valores como cantidad y subtotal dependen de la instancia específica de la línea de pedido.
3FN (Tercera FN)	Cumple con la 2FN y no existen dependencias funcionales transitivas entre atributos no clave.	La entidad categoria se independiza de producto, evitando la redundancia y desincronización de descripciones.

3. Restricciones e Integridad de Datos

El motor relacional aplica políticas de integridad para preservar la exactitud, coherencia y validez de la información almacenada:

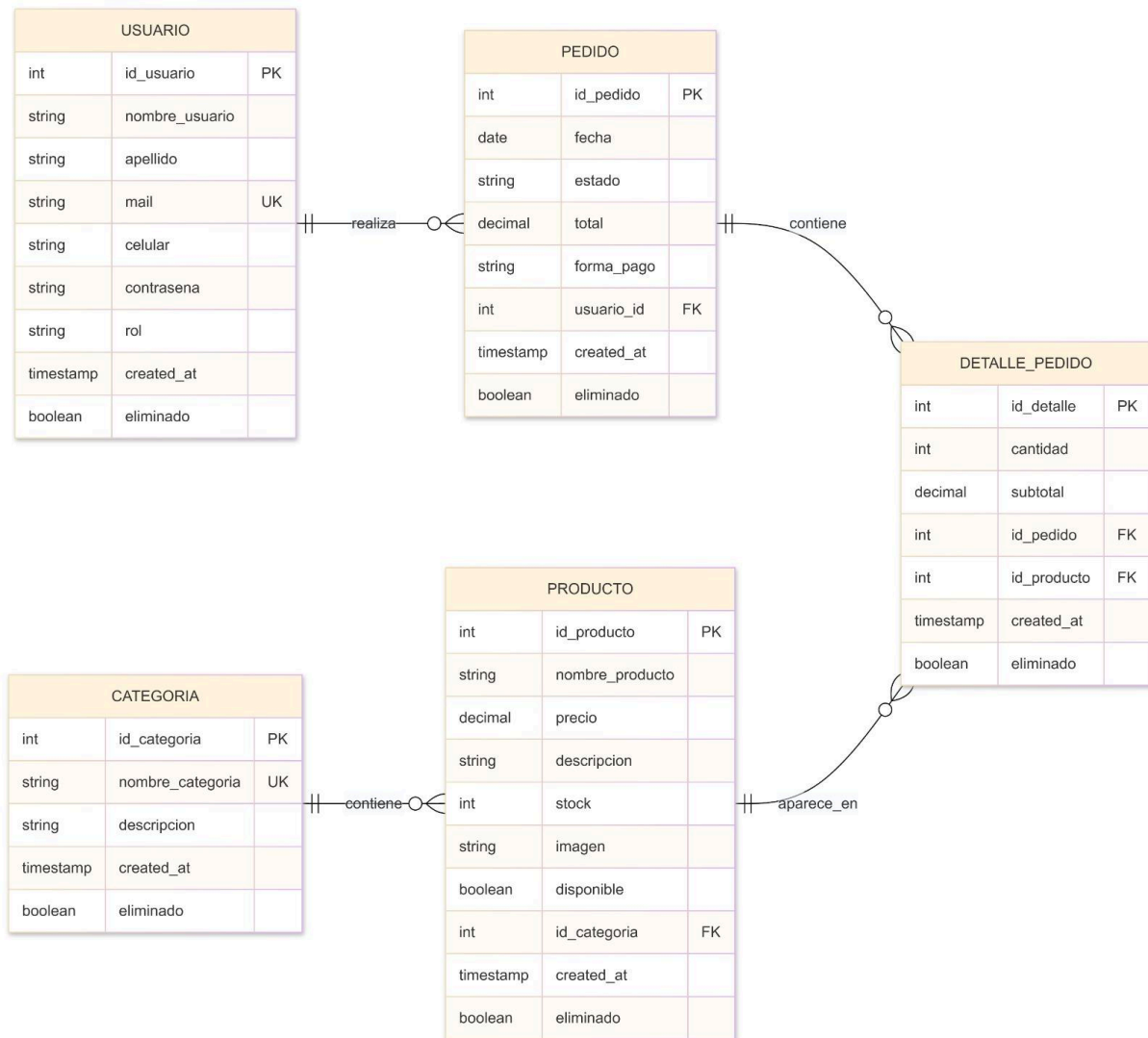
- **Integridad de Entidad:** Garantizada mediante identificadores únicos primarios (BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY), asegurando registros únicos e irrepetibles.
- **Integridad Referencial:** Controlada a través de claves foráneas (REFERENCES) con cláusulas restrictivas (ON DELETE RESTRICT), impidiendo registros huérfanos en pedidos y detalles.
- **Integridad de Dominio:** Implementación de tipos de datos estrictos, tipos enumerados (rol, estado_pedido, forma_pago) y validaciones explícitas CHECK (precio ≥ 0 , stock ≥ 0 , cantidad > 0).
- **Baja Lógica vs. Baja Física:** Estrategia de borrado lógico mediante la columna eliminado BOOLEAN, protegiendo el historial contable y la trazabilidad de transacciones previas.

4. Transacciones y Propiedades ACID

Una transacción es una secuencia indivisible de operaciones tratada como una única unidad lógica de trabajo. El sistema garantiza el estándar **ACID**:

- **Atomicidad (A):** Ejecución bajo el criterio de "todo o nada". En el procedimiento sp_crear_pedido, si ocurre un fallo o validación inválida, la transacción completa se revierte (ROLLBACK), evitando estados parciales o inconsistentes.
- **Consistencia (C):** Asegura que la base de datos transicione únicamente de un estado válido a otro, cumpliendo todas las reglas de negocio y restricciones relacionales.
- **Aislamiento (I):** Bloqueo pesimista a nivel de fila mediante SELECT ... FOR UPDATE sobre las tuplas de producto para impedir condiciones de carrera y sobreventa en operaciones concurrentes.
- **Durabilidad (D):** Una vez confirmada la operación (COMMIT), las modificaciones registradas en pedidos y stock persisten permanentemente, incluso ante fallas críticas del sistema.

Diagrama



Preguntas guía.

- ¿Por qué la relación entre producto y pedido no puede resolverse como una relación binaria simple 1:N? ¿Qué información se perdería si lo intentaras?
- ¿Qué entidad tiene participación parcial en la relación con categoría, y cuál tiene participación total? ¿Qué pasaría si invirtieras esa lectura?
- ¿Alguno de tus atributos podría descomponerse en partes más simples (por ejemplo, un nombre completo en nombre y apellido)? Justifica tu decisión de dejarlo compuesto o de separarlo.

- 1) Si se pone la clave foránea en PRODUCTO (1 Pedido a N Productos): Cada producto sólo podría pertenecer a un único pedido en toda la historia del sistema. Al comprarse una vez, quedaría bloqueado para futuras ventas.

Si se pone la clave foránea en PEDIDO (1 Producto a N Pedidos): Cada pedido solo podría contener un único tipo de producto. Sería imposible armar un "carrito" con múltiples artículos distintos.

- 2) PRODUCTO tiene participación total, ya que no pueden existir productos sin una categoría asignada. En cambio CATEGORÍA tiene participación parcial, ya que pueden existir categorías recién creadas o vacías que todavía no tengan un producto asociado.

- 3) Podría descomponerse dirección en: calle, numero, piso_depto, ciudad_localidad, provincia_estado, codigo_postal.

Esto permite calcular costos y tiempos de envío según la zona o código postal. Facilita la validación formal de direcciones y la integración con APIs de logística/couriers y permite hacer análisis estadísticos y filtros de ventas por región o localidad.

Decisiones de Diseño

Arquitectura y estrategias aplicadas en la base de datos FoodStore

1. Normalización y Descomposición Relacional

- **Alcance hasta 3FN:** El esquema fue normalizado hasta la **Tercera Forma Normal (3FN)** para erradicar la redundancia y prevenir anomalías de inserción, actualización y borrado. Se separó la entidad categoria de producto para evitar inconsistencias en nombres y descripciones.
- **Resolución de Relación N:M:** La relación muchos a muchos entre pedido y producto se materializó a través de la tabla intermedia detalle_pedido, la cual contiene una restricción de unicidad compuesta UNIQUE (id_pedido, id_producto) para evitar ítems duplicados en una misma orden.
- **Inmutabilidad de Precios Históricos:** En detalle_pedido se almacena una copia explícita del precio_unitario y subtotal al momento de la compra. Esto desacopla el registro contable de futuras variaciones de precios en la tabla producto, garantizando la validez histórica de las ventas.

2. Manejo del Total y Cálculo Automático

- ---

Desnormalización Controlada del Total: La columna total se mantiene en la tabla pedido para optimizar consultas frecuentes de listado y facturación (evitando agregaciones costosas SUM() en cada lectura masiva).
- **Integridad Automatizada vía Triggers:**
 - **trg_subtotal (BEFORE INSERT/UPDATE):** Completa automáticamente el precio_unitario tomando el valor actual de producto si no fue provisto, y calcula el subtotal = cantidad * precio_unitario a nivel de fila.
 - **trg_total_ins / trg_total_upd (AFTER STATEMENT):** Utilizan tablas de transición (REFERENCING NEW TABLE AS afectados) para recalcular el total del pedido mediante la función calcular_total_pedido() en una única pasada por sentencia, maximizando el rendimiento en inserciones por lote (*bulk inserts*).

3. Borrado Lógico (Soft Delete)

- ---

Preservación del Histórico y Trazabilidad: En lugar de borrados físicos destructivos (DELETE), todas las entidades implementan la columna eliminado BOOLEAN NOT NULL DEFAULT FALSE.
- **Protección de Integridad Referencial:** Evita errores de clave foránea (ON DELETE RESTRICT) o registros huérfanos. Si un usuario o producto se da de baja del catálogo activo, los pedidos históricos facturados siguen siendo válidos y consultables.
- **Abstracción con Vistas:** Se implementaron vistas especializadas (v_productos_vigentes, v_pedidos_resumen, v_categorias_vigentes) que encapsulan el filtro WHERE eliminado = FALSE, simplificando las consultas de la capa de aplicación y reduciendo el riesgo de errores humanos.

4. Estrategia de Indexación

Para optimizar los planes de ejecución y acelerar las consultas más críticas, se crearon los siguientes índices:

- **Índices en Claves Foráneas (idx_producto_id_categoria, idx_id_pedido_usuario):** PostgreSQL no indexa automáticamente las claves foráneas. Estos índices aceleran drásticamente las operaciones de combinación (JOIN) entre productos-categorías y pedidos-usuarios.
- **Índice Parcial (idx_producto_nombre):** Definido con la cláusula WHERE eliminado = FALSE. Al indexar únicamente los productos activos, se reduce el tamaño del árbol B-Tree en disco, optimizando la búsqueda por catálogo y reduciendo el costo de mantenimiento del índice.
- **Índice Temporal (idx_pedido_fecha):** Diseñado para acelerar los filtros por rango de fechas, generación de reportes mensuales y análisis de facturación temporal.

Dificultades, Soluciones y Referencias

Documentación complementaria de implementación y entregables del proyecto FoodStore

1. Dificultades Encontradas y Soluciones Implementadas

Dificultad / Desafío Técnico	Solución Implementada
Recálculo del total de pedidos en inserciones masivas Al insertar múltiples detalles en una sola sentencia, recalculaba el total fila por fila mediante triggers FOR EACH ROW generaba sobrecarga y bloqueos innecesarios.	Se implementaron triggers a nivel de sentencia (FOR EACH STATEMENT) utilizando tablas de transición (REFERENCING NEW TABLE AS afectados) para procesar y actualizar todos los pedidos involucrados en un único paso eficiente.
Concurrencia y riesgo de sobreventa (Race Conditions) En compras simultáneas sobre productos con bajo stock, existía el riesgo de vender más unidades de las disponibles físicamente.	Se encapsuló la lógica en el procedimiento almacenado sp_crear_pedido aplicando un bloqueo pesimista a nivel de fila con SELECT ... FOR UPDATE antes de validar y descontar el stock dentro de la misma transacción.
Pérdida de precios históricos tras variaciones de catálogo Si el precio de un producto cambiaba en la tabla maestra, se alteraban retrospectivamente los totales de compras previas.	Se definió una copia inmutable del precio_unitario y subtotal en detalle_pedido, congelando el precio mediante el trigger trg_subtotal al momento exacto de la compra.
Mantenimiento de integridad referencial con bajas de datos El borrado físico (DELETE) de productos o usuarios con historial rompía las restricciones de clave foránea o borraba transacciones pasadas.	Se implementó un patrón de borrado lógico (Soft Delete) con la bandera eliminado BOOLEAN y se crearon vistas especializadas (v_productos_vigentes, etc.) para abstraer los filtros activos en la capa de consumo.

2. Bibliografía y Webgrafía

- **Silberschatz, A., Korth, H. F., & Sudarshan, S.** (2020). *Database System Concepts* (7ma ed.). McGraw-Hill. (Conceptos de modelo relacional, normalización y propiedades ACID).
- **PostgreSQL Global Development Group.** (2024). *PostgreSQL Documentation: PL/pgSQL - SQL Procedural Language, Triggers & Concurrency Control*. Recuperado de la documentación oficial de PostgreSQL.
- **Elmasri, R., & Navathe, S. B.** (2016). *Fundamentals of Database Systems* (7ma ed.).

Pearson. (Diseño conceptual, lógico e integridad referencial).

- **Codd, E. F.** (1970). *A Relational Model of Data for Large Shared Data Banks*. Communications of the ACM, 13(6), 377-387.

3. Enlaces al Repositorio y Video Demostrativo

A continuación se detallan los enlaces correspondientes al código fuente, scripts de despliegue y video explicativo del proyecto:

- **Repositorio en GitHub:**
- **Video de Demostración / Defensa:**